

二. STC51单片机阶段

1. 学习定位总结

把本节内容涉及到面试官提问的标准答案背诵下来！！！！！！！！如果背诵内容难以理解，请到对应课程章节复习视频教程。

投递简历，等待面试期间，不要过于焦虑，找工作中的等待就是一个必然过程，静下新来，把课程温习一遍，不用动手，可以1.5倍速观看课程，

看完后，花1-2天时间，仔细看这个文档，非常重要。

1.1 为什么要学习C51单片机-如果问

目前嵌入式流行的芯片是传统的STM32和ESP32，而STM32也封装了大量的HAL库和标准库，ESP32更是集成了FreeRTOS的使用方案，都不能很好的

去学习关于寄存器及芯片手册时序图分析的内容。STC89C52RC虽然有点老，性能低端只有普通IO，中断，寄存器，UART的硬件特性，由于开发环境

没有带相关SDK，都要自己实现，很好锻炼了新手对芯片手册中寄存器的阅读能力，对时序图的分析能力。

有了这个基础之后，如果后期学习STM32标准库HAL库到寄存器级别，就能有相应的能力去做代码分析，甚至对高端ARM架构，能跑Linux的芯片手册也有帮助。

在学习完C语言之后，C51是新手最佳的关于硬件底层学习的芯片，没有之一。

2. 定时器总结

2.1 定时器的功能

1. **普通定时器**：用于中断定时、波特率发生等。
2. **计数器**：用于测量脉冲宽度，实现测频功能。

2.2 定时器C51相关寄存器

STC51单片机的定时器/计数器工作模式由TMOD寄存器控制。

TMOD是一个8位寄存器，分别控制着定时器0和定时器1。

GATE位控制定时器的开启与停止，C/T位决定定时器是工作在定时器模式还是计数器模式，

- M1和M0位用于选择定时器/计数器的计数模式
- 打开定时器0中断ET0=1，还要打开总中断EA=1
- 定时器初值TL0, TH0
- 开始计时 TR0 = 1

2.3 定时器的应用

定时器用于时间管理，用于事件计数，用于外设交互，用于软件辅助，课程都有涉及开发案例，针对相应内容预测面试官会问的

问题，并标准化答案，大家可能会做，也做过，难免会忘记，或者不会表达，请背诵本节粗体字答案。

2.3.1 时间管理-SG90PWM

提供时序参考，管理时间事件。比如课程中关于用定时器来结合基本IO口输出PWM波形，要去复习如何控制舵机的角度

```
#include "reg52.h"

sbit sg90_con = P1^1;
int jd;
int cnt = 0;

void Delay2000ms()    //@11.0592MHz
{
    // ...
}

void Time0Init()
{
    //1. 配置定时器0工作模式位16位计时
    TMOD = 0x01;
    //2. 给初值，定一个0.5出来
    TL0=0x33;
    TH0=0xFE;
    //3. 开始计时
    TR0 = 1;
    TF0 = 0;
    //4. 打开定时器0中断
    ET0 = 1;
    //5. 打开总中断EA
    EA = 1;
}

void Delay300ms()    //@11.0592MHz
{
    //...
}

void main()
{
    Delay300ms(); //让硬件稳定一下
    Time0Init(); //初始化定时器
    jd = 1;      //初始角度是0度，0.5ms,溢出1就是0.5，高电平
    cnt = 0;
    sg90_con = 1; //一开始从高电平开始

    //每隔两秒切换一次角度
    while(1){
        jd = 3; //90度 1.5ms高电平
        cnt = 0;
        Delay2000ms();
        jd = 1; //0度
        cnt = 0;
        Delay2000ms();
    }
}
```

```

}

void Time0Handler() interrupt 1
{
    cnt++; //统计爆表的次数。cnt=1的时候，报表了1
    //重新给初值
    TL0=0x33;
    TH0=0xFE;

    //控制PWM波
    if(cnt < jd){
        sg90_con = 1;
    }else{
        sg90_con = 0;
    }

    if(cnt == 40){ //爆表40次，经过了20ms
        cnt = 0; //当100次表示1s，重新让cnt从0开始，计算下一次的1s
        sg90_con = 1;
    }
}
}

```

面试题，如果通过定时器来控制角度的？？？

SG90舵机通过接收PWM（脉冲宽度调制）信号来控制舵机的转动角度。

PWM信号的周期通常为20ms，而高电平的宽度则在0.5ms至2.5ms之间变化。

这个高电平宽度与舵机的输出角度呈线性关系，即高电平宽度越长，舵机转动的角度越大

在20ms的周期内，高电平的持续时间决定舵机角度。

在主函数中修改用于角度控制的全局变量，比如jd变量，

只要在定时器中断处理函数中把jd变量来影响PWM波形中20ms周期内高电平的时间，就可以控制角度。

《以上背诵》

2.3.2 事件计数-小车测速

用于脉冲测量、频率计数等。比如课程中关于小车速度的测试

```

#include "motor.h"
#include "reg52.h"

extern unsigned int speedCnt;
unsigned int speed;
char signal = 0;
unsigned int cnt = 0;

void Time0Init()
{
    //1. 配置定时器0工作模式位16位计时
    TMOD = 0x01;
    //2. 给初值，定一个0.5出来
    TL0=0x33;
}

```

```

    TH0=0xFE;
    //3. 开始计时
    TR0 = 1;
    TF0 = 0;
    //4. 打开定时器0中断
    ET0 = 1;
    //5. 打开总中断EA
    EA = 1;
}

void Time0Handler() interrupt 1
{
    cnt++; //统计爆表的次数。cnt=1的时候，报表了1
    //重新给初值
    TL0=0x33;
    TH0=0xFE;

    if(cnt == 2000){//爆表2000次，经过了1s
        signal = 1;
        cnt = 0; //当100次表示1s，重新让cnt从0开始，计算下一次的1s
        //计算小车的速度，也就是拿到speedCnt的值
        speed = speedCnt;
        speedCnt = 0;//1秒后拿到speedCnt个格子，就能算出这1s的速度，格子清零
    }
}
}

```

面试官问，你是如何进行测速的？？？

测速是通过小车轮子上的码盘，当车轮滚动一圈，码盘会触发一次外部中断，记录滚动圈数在全局变量中

使用定时0做一个1秒的准确时间，代码设计定时器0每个500微秒就触发一次中断，在定时器中断处理函数中检测是否满足2000次中断，

如果触发2000次中断，说明经过了1秒，读取全局变量圈数，那么速度就等于轮子周长*圈数再除以1秒，得出cm/s的数据

《以上背诵》

2.3.3 与外设交互-超声波模块

配合外设进行时序控制，比如课程中使用定时器来配合超声波模块进行测距，以下是课程中超声波测距的代码

```

#include "reg52.h"

//距离小于10cm,D5亮，D6灭，反之相反现象

sbit D5 = P3^7;//根据原理图（电路图），设备变量led1指向P3组IO口的第7口
sbit D6 = P3^6;//根据原理图（电路图），设备变量led2指向P3组IO口的第6口
sbit Trig = P1^5;
sbit Echo = P1^6;

void Delay10us() // @11.0592MHz
{

```

```

    unsigned char i;

    i = 2;
    while (--i);
}

void Time0Init()
{
    TMOD &= 0xF0;          //设置定时器模式
    TMOD |= 0x01;
    TH0 = 0;
    TL0 = 0;
    //设置定时器0工作模式1，初始值设定0开始数数，不着急启动定时器
}
/*
十进制2左移1位，变成20。相当于乘以10
二禁止1左移1位，变成10（2）。相当于乘以2，左移8位，乘以2的8次方=256；*/

void startHC()
{
    Trig = 0;
    Trig = 1;
    Delay10us();
    Trig = 0;
}

void main()
{
    double time;
    double dis;

    Time0Init();

    while(1){
        //1. Trig ，给Trig端口至少10us的高电平
        startHC();
        //2. echo由低电平跳转到高电平，表示开始发送波
        while(Echo == 0);
        //波发出去的那一下，开始启动定时器
        TR0 = 1;
        //3. 由高电平跳转回低电平，表示波回来了
        while(Echo == 1);
        //波回来的那一下，我们开始停止定时器
        TR0 = 0;
        //4. 计算出中间经过多少时间
        time = (TH0 * 256 + TL0)*1.085;//us为单位
        //5. 距离 = 速度（340m/s）* 时间/2
        dis = time * 0.017;
        if(dis < 10){
            D5 = 0;
            D6 = 1;
        }else{
            D5 = 1;
            D6 = 0;
        }
        //定时器数据清零，以便下一次测距
        TH0 = 0;
    }
}

```

```

        TLO = 0;
    }
}

```

面试官问，如何实现超声波测距的????

超声波模块开发有根据一个时序图来做的，主要包含超声波启动信号，比如给10us的高电平，

这里用软件延时制作出10s的时间，说白了就是用循环函数空消耗

超声波信号发出到信号返回的时间，很重要，这个时间的计算使用定时器实现的，设置定时器0工作模式1，初始值设定0开始计时

不断检测Echo引脚的信号，如果Echo引脚有低电平跳变到高电平，说明信号返回，停止定时器并读取定时器寄存器中的值，

在通过距离=速度*时间的计算公式，得出前方障碍物距离。

《以上背诵》

2.3.4 软件辅助-精准定时

实现各种复杂的时间相关的软件逻辑，比如精确定时出1秒的时间。

```

#include "reg52.h"

sbit led = P3^6;
sbit led1 = P3^7;
int cnt = 0;

void Time0Init()
{
    //1. 配置定时器0工作模式位16位计时
    TMOD = 0x01;
    //2. 给初值，定一个10ms出来
    TLO=0x00;
    TH0=0xDC;
    //3. 开始计时
    TR0 = 1;
    TF0 = 0;
    //4. 打开定时器0中断
    ET0 = 1;
    //5. 打开总中断EA
    EA = 1;
}

void main()
{
    led = 1;
    Time0Init();
    while(1){
    }
}

void Time0Handler() interrupt 1
{
    cnt++; //统计爆表的次数
}

```

```

//重新给初值
TL0=0x00;
TH0=0xDC;
if(cnt == 100){//爆表100次，经过了1s
    cnt = 0;    //当100次表示1s，重新让cnt从0开始，计算下一次的1s
    led = !led;//每经过1s，翻转led的状态
}
}

```

面试官问，如何精确定义出1s的时间？？？

51单片机因为是16位的定时计数器，没有办法直接定义出1秒的时间，实现的策略是精准定出10ms的时间，

每经过10ms就引发定时器中断，在中断函数中计算中断发生次数，累计中断发生次数100次，就获得1秒的精确定时效果。

《以上背诵》

3.4 定时器编程指南

- **初始化：**对TMOD寄存器赋值，确定T0和T1的工作方式。
- **设置初值：**计算初值，并将其写入THx和TLx寄存器。
- **中断开发：**如果使用中断方式，对IE寄存器赋值开发中断。
- **启动定时器：**使TRx（x为0或1）置位，启动定时/计数器。

3.5 定时器的工作模式：

面试官问的话，就回答**有4种工作模式，比如13位定时计数，16位定时计数，具体的要看芯片手册。**

51单片机的定时器共有四种工作模式，具体如下：

1. 模式0：13位定时器/计数器 (TMOD = 0x00)

- 工作方式：仅作为定时器，自动重载。在这种模式下，定时器/计数器实际上是一个13位的计数器，最高位是溢出标志位，不计数。
- 工作情景：用于周期性的定时任务，产生固定时间间隔的中断。

2. 模式1：16位定时器/计数器 (TMOD = 0x01)

- 工作方式：仅作为定时器，手动重载。这种模式下，定时器/计数器是一个完整的16位计数器。
- 工作情景：用于需要更长时间跨度的定时任务，由软件手动控制定时器的重装值。

3. 模式2：8位自动重装定时器/计数器 (TMOD = 0x02)

- 工作方式：作为定时器，并在中断处理程序中自动重装初值。这种模式下，定时器/计数器是一个8位计数器，每次溢出后会自动重装初值。
- 工作情景：适用于需要周期性定时任务，且中断处理程序需要花费较长时间。

4. 模式3：双重8位定时器/计数器 (TMOD = 0x03)

- 工作方式：两个8位定时器/计数器一起工作。在这种模式下，定时器0被分为两个独立的8位定时器/计数器。
- 工作情景：适用于需要同时进行两个定时任务或频率测量的情况。

每种模式都有其特定的应用场景，开发者可以根据实际需求选择合适的工作模式来配置定时器/计数器。

3. UART总结

3.1 UART协议--重要！

UART，**用于异步通信**。UART协议允许两个设备之间通过单一的通信线路进行数据传输。以下是UART协议的一些关键特性：

- 1. **异步通信：**
 - UART通信是异步的，意味着发送方和接收方的时钟是独立的，不需要同步信号。
- 2. **数据格式：**
 - UART通信的数据格式通常包括数据位、起始位和停止位。
 - **起始位：**每个字节的开始是一个低电平信号（0）。
 - **数据位：**可以是5到9位，常用的是8位。
 - **奇偶校验位（Parity Bit）：**可选的，用于错误检测。
 - **停止位：**数据字节后的一个或两个高电平信号（1），表示数据传输的结束。
- 3. **波特率：**
 - 波特率（Baud Rate）是UART通信的速率，即每秒传输的信号单位数。常见的波特率有9600、19200、38400、115200等。
- 4. **全双工通信：**
 - UART支持全双工通信，即可以同时进行数据的发送和接收。
- 5. **硬件连接：**
 - 通常只需要两条线：TX（发送线）和RX（接收线）。

3.2 常用寄存器

以下是STC51单片机中与UART通信相关的常用寄存器

寄存器名称	功能描述
SCON	串行控制寄存器，用于设定串行口的工作方式、接收/发送控制以及设置状态标志等。
PCON	电源控制寄存器，包括波特率选择和帧错误控制位。
SBUF	串行数据缓冲寄存器，物理上是两个独立的寄存器，但占用相同的地址。
TMOD	定时器模式寄存器，用于配置定时器的的工作模式，UART通信中用于控制波特率。
TH1	定时器1高8位，用于设置定时器1的初值，进而控制波特率。
TL1	定时器1低8位，用于设置定时器1的初值，进而控制波特率。

3.3 代码回顾

```
#include "reg52.h"
#include "intrins.h"

sfr AUXR = 0x8E;

void UartInit(void)    //9600bps@11.0592MHz
{
    AUXR = 0x1;
    SCON = 0x40; //选择串口工作方式1
    TMOD &= 0x0F;
    TMOD |= 0x20; //定时器1工作在8位自动重载

    TH1 = 0xFD;
    TL1 = 0xFD; //9600波特率根据公式算出的初始值
    TR1 = 1; //定时器开始工作!
    // ES = 1;
    // EA = 1;
}

void Delay1000ms()    //@11.0592MHz
{
    unsigned char i, j, k;

    _nop_();
    i = 8;
    j = 1;
    k = 243;
    do
    {
        do
        {
            while (--k);
        } while (--j);
    } while (--i);
}

void main()
{
    char data_msg = 'a';

    //配置C51串口的通信方式
    UartInit();

    while(1){
        Delay1000ms();
        //往发送缓冲区写入数据，就完成数据的发送
        SBUF = data_msg;
    }
}
```

3.4 面试常见问题

3.4.1 简述一下uart如何传输一个字符A

UART传输一个字符'A'的过程如下：

首先，单片机通过配置UART的相关寄存器（如SCON、TMOD、TH1等）设置波特率和通信参数。

然后，将字符'A'的ASCII码值（65）写入到UART的数据缓冲寄存器SBUF中。

UART硬件检测到SBUF非空后，自动在起始位后发送'A'的8位ASCII码数据，（**从最低位（LSB）开始发送，然后逐步发送到最高位（MSB）**）

接着发送一个可选的奇偶校验位，最后发送一个或两个停止位来标识传输结束。

接收方的UART在检测到起始位后开始接收数据，并在停止位后确认字符'A'已完全接收。

3.4.2 简述一下uart通信协议

UART通信协议是一种异步串行通信机制，

它允许设备之间通过两条线（发送线TX和接收线RX）进行数据交换，数据以字符为单位传输，

每个字符由起始位、一定数量的数据位（通常是5到8位）、可选的奇偶校验位和停止位组成，

所有这些位都以特定的波特率（即每秒传输的符号数）进行传输，从而实现设备间的同步通信。

3.4.3 如何提升串口通信效率

提升串口通信效率可以通过优化通信参数、使用中断或DMA（直接内存访问）技术减少CPU占用、

采用更高的波特率以加快数据传输速度、减少不必要的通信握手和校验过程、

合理配置缓冲区大小以平衡内存使用和数据吞吐量，以及在软件层面优化协议栈和数据处理流程来实现。

3.4.4 C51和STM32对uart使用上的区别

能记得下的话，可以去记住

C51单片机和STM32单片机在使用串口（UART）时的主要区别体现在以下几个方面：

1. 硬件架构：

- C51单片机基于传统的8051架构，通常只包含一个UART接口。
- STM32单片机基于ARM Cortex-M系列处理器，拥有更复杂的总线结构和更多的串口资源，如USART1、USART2、USART3等。

2. 外设功能：

- C51单片机的外设种类较少，功能相对简单。
- STM32单片机集成了丰富的片上外设，包括多个UART接口，以及其他高级通信接口如SPI、I2C等。

3. 波特率配置：

- C51单片机的波特率配置通常涉及到定时器的设置，需要手动计算并设置波特率生成器。
- STM32单片机的波特率配置更为灵活，可以通过软件配置APB时钟和USART分频寄存器来设置。

4. 编程和配置：

- C51单片机的串口配置相对简单，主要涉及SCON、PCON和TMOD等寄存器的设置。

- STM32单片机的串口配置更为复杂，需要配置GPIO、RCC（时钟控制）、USART等多个寄存器，并且可以使用CubeMX等软件工具来简化配置。

5. 中断和DMA：

- C51单片机的UART通信通常使用中断方式处理数据接收和发送，DMA功能较弱。
- STM32单片机支持更高级的中断处理和DMA数据传输，可以提高数据传输效率。

6. 内存和处理能力：

- C51单片机通常具有较小的内存容量和较低的处理能力。
- STM32单片机具有更大的内存容量和更高的处理能力，适合处理更复杂的数据和通信任务。

3.4.5 你用串口做过哪些开发

在我之前的开发经验中，串口主要用于网络通信模块和语音模块，网络模块较多是用AT指令驱动，比如wifi模块，蓝牙模块，4g模块，还有物联网NB-IOT模块等

语音模块使用su-06，也是通过串口数据来交互模块中的固件，通过发送特定串口数据来播报语音，或者根据接收的串口数据来判断语音模块识别结果。

3.4.6 小车中关于串口的使用

智能小车属于大学期间的比赛项目，当时串口使用有两个目的，第一个目的是调试，比如驱动电机，获取电机速度，获取小车前方障碍物距离等，

另外一个目的是连接语音模块，根据语音模块的识别结果来切换小车的工作模式，比如避障模式，跟随模式，手机控制模式的切换。

因为51单片机只有一个串口，在项目设计中把一个串口拆分两个模块连接，比如语音模块只要语音识别功能的时候，就连接到c51串口的输入口，

需要通过蓝牙或者wifi把小车数据比如向上位机发送车速，发送温湿度采集数据，那么只要连接到C51串口的输出口。

4. IIC总结

4.1 IIC协议概述

C51单片机实现I2C通信主要涉及到以下几个方面：

1. **I2C总线结构**：I2C通信使用两条线，**数据线（SDA）和时钟线（SCL）**，所有设备共享这两条线。
2. **通信时序**：I2C通信时序包括**起始信号、数据传输、应答信号和停止信号**。主设备生成时钟信号并控制通信的开始和结束，从设备响应时钟信号并进行数据的发送或接收。
3. **数据传输协议**：I2C数据传输包括发送和接收字节，每个字节包含8位，**从最高位开始传输**。发送时，主机将数据位放在SDA线上，然后拉高SCL，从机在SCL高电平期间读取数据位。接收时，从机将数据位放在SDA线上，主机在SCL高电平期间读取数据位。
4. **软件模拟I2C通信**：C51单片机可以通过软件模拟I2C通信时序，包括起始信号、发送字节、接收字节、发送应答和接收应答等。
5. **硬件接口**：C51单片机通过两个引脚（如P2[^]1和P2[^]0）分别连接到I2C的SCL和SDA，实现与I2C设备的通信。
6. **编程实现**：在C51单片机上，需要编写程序来控制I2C通信，包括发送起始信号、发送数据字节、接收数据字节、发送和接收应答位以及发送停止信号。

7. **应用示例**：例如，使用C51单片机与oled进行通信时，可以通过编写特定的I2C通信协议函数来实现数据的读写操作。

总结来说，C51单片机实现I2C通信需要理解I2C的总线结构和通信时序，并通过软件编程来模拟I2C通信协议，控制硬件接口进行数据的发送和接收。

4.2 代码回顾

```
#include "reg52.h"
#include "intrins.h"

sbit scl = P0^1;
sbit sda = P0^3;

void IIC_Start()
{
    sda = 1;
    scl = 1;
    _nop_();
    sda = 0;
    _nop_();
}

void IIC_Stop()
{
    sda = 0;
    scl = 1;
    _nop_();
    sda = 1;
    _nop_();
}

char IIC_ACK()
{
    char flag;
    sda = 1; //就在时钟脉冲9期间释放数据线
    _nop_();
    scl = 1;
    _nop_();
    flag = sda;
    _nop_();
    scl = 0;
    _nop_();

    return flag;
}

void IIC_Send_Byte(char dataSend)
{
    int i;

    for(i = 0; i < 8; i++){
        scl = 0; //scl拉低，让sda做好数据准备
        sda = dataSend & 0x80; //1000 0000获得dataSend的最高位，给sda
        _nop_(); //发送数据建立时间
```

```

        scl = 1; //scl拉高开始发送
        _nop_(); //数据发送时间
        scl = 0; //发送完毕拉低
        _nop_(); //
        dataSend = dataSend << 1;
    }
}

void main()
{
    int a = 10;

    IIC_Start();

}

```

4.3 面试官常问

4.3.1 说一下IIC通信协议

I2C通信协议是一种多主机、同步、串行计算机总线，用于连接微控制器和其他设备，通过两条线（数据线SDA和时钟线SCL）实现数据的传输，其特点是支持多个设备共享同一总线，且通信时序由主设备控制，从设备响应，包括起始条件、数据传输、应答位和停止条件等关键步骤，广泛应用于微控制器与外围设备如传感器、存储器等之间的数据交换。

4.3.2 描述IIC的起始信号，终止信号和停止信号

在I2C通信协议中，起始信号由SDA线在SCL保持高电平时从高电平跳变到低电平产生，表示一次新的数据传输开始；

停止信号则是SDA线在SCL保持高电平时从低电平跳变到高电平，标志着数据传输的结束；

而重复启动信号（Restart Condition）是SDA线在SCL为高电平时从高电平跳变到低电平，紧接着在SCL为低电平时回到高电平，

用于在不释放SDA和SCL线的情况下开始新一轮的数据传输。

4.3.3 描述IIC总线数据传输过程

I2C总线数据传输过程开始于主设备生成的起始信号，随后主设备通过SDA线发送数据字节，

每个字节后跟一个由从设备在SCL线产生的应答信号（ACK），数据传输可以包含多个字节，

并通过发送应答位来确认每个字节的成功接收，传输结束后，主设备发出停止信号或重复启动信号来结束当前传输或开始新的传输周期。

在整个过程中，SCL线由主设备控制，用于同步数据的发送和接收。

4.3.4 IIC通信有几种工作模式

I2C通信协议主要有以下几种工作模式：

1. **单主单从模式**：在这种模式下，只有一个主设备和一个从设备。主设备通过发送从设备地址选择具体的从设备进行通信。
2. **单主多从模式**：在这种模式中，主设备可以与多个从设备通信。每个从设备都有唯一的地址，主设备通过发送从设备地址选择特定的从设备。

3. **多主多从模式**：在这种模式中，多个主设备可以与多个从设备通信。I2C协议通过仲裁机制确保只有一个主设备在总线上进行数据传输。

这些模式允许I2C总线在不同的应用场景中灵活地进行数据通信。

4.3.5 IIC通信总线上拉电阻多大

I2C总线的上拉电阻通常推荐值在1kΩ到10kΩ之间。

对于标准速率为100kHz的I2C通信，常见的选择是4.7kΩ或10kΩ。

对于快速模式（400kHz）或更快的高速模式（3.4MHz），可能需要降低阻值到几千欧姆，以保证信号的快速上升沿。

在实际应用中，常见的I2C上拉电阻阻值范围从1kΩ到10kΩ

4.3.6 C51如何通过IIC驱动OLED屏的

这里设计OLED屏的数据手册，里面集成很多控制命令，包括初始化，位置信息，显示内容等

在C51单片机上通过I2C驱动OLED显示屏，主要涉及以下几个步骤：

初始化I2C通信：首先需要定义I2C通信所需的引脚，并初始化这些引脚的状态。

例如，将P1^0设置为I2C时钟线（SCLK），P1^1设置为I2C数据线（SDIN）

编写I2C通信函数：包括I2C的起始信号、停止信号、字节写入、等待应答等函数。

这些函数通过软件模拟I2C协议的时序，控制数据线和时钟线来实现数据的发送和接收

发送命令和数据：通过编写的I2C通信函数，向OLED发送控制命令和显示数据。通常，命令和数据需要分别发送，

命令用于设置OLED的工作模式，数据用于显示内容

OLED显示控制：定义OLED的显示控制函数，

如清屏、显示字符、显示数字、显示字符串等。这些函数内部会调用I2C通信函数来发送具体的命令和数据

5. 常用外设模块总结

5.1 C51如何获取DHT11的数据

C51单片机获取DHT11温湿度数据的过程涉及初始化通信、发送唤醒信号、接收数据以及数据校验。

首先，C51单片机通过一个I/O口与DHT11的数据线连接，并确保电源和地线连接正确。

然后，通过发送一个低电平持续至少18毫秒的唤醒信号来启动DHT11，随后拉高数据线至少20微秒以完成启动流程。

DHT11响应后，会发送40位的数据，包括湿度和温度的整数与小数部分，以及一个校验和。

C51单片机通过检测数据线上的电平变化来读取这些数据位，并进行校验以确保数据的准确性。

最后，将接收到的原始数据转换为实际的温度和湿度值，通常温度数据需要除以10来得到摄氏度数值，湿度数据同样需要除以10来得到百分比数值。

5.2 C51如何使用Lcd1602

C51单片机使用LCD1602显示屏时，需要通过并行数据线（如P0口）和控制线（如RS、RW和E）与LCD进行连接，

然后编写程序来初始化LCD（设置数据长度、显示行数、显示模式等）

，之后通过发送命令和数据来控制LCD显示文本、数字或其他字符，其中命令用于配置LCD的操作模式，而数据则用于在LCD上显示具体的字符或清屏等操作。

现在我只能根据之前编写调试过的代码来回忆大概流程，具体的细节还是要在实际开发过程中看数据手册才能搞定。

6. C51智能小车定位及面试问答

6.1 综合介绍

俗话说，每个嵌入式工程师都躲不开一辆智障小车。

51小车是我刚学习嵌入式时候做的东西，在学校为了蓝桥杯准备的。

实现的功能由小车方向控制，测速，舵机摇头避障，跟随，语音控制，上位机控制等，

现在看来，做小车不是目的，也无法支撑我作为面试入行的有力项目，但是这个小车把单片机的基本IO口，中断，定时器，串口，模拟IIC等做了比较综合的

应用，对于刚学习软硬件开发的时候，对我也有比较大的帮助。

6.2 功能实现介绍

电机控制使用的是L298N驱动，输入电压7v-12v，根据一对高低电平来控制电机正反转，比较简单，这里实现了速度控制，使用C51定时器来模拟出PWM信号，

用于寻迹时候的转弯控制。寻迹功能使用的是红外寻迹模块，模块经过黑线或者白线反馈的数据是高低电平，这样一边一个寻迹模块就可以来控制小车转向。

完成寻迹功能，以上两个功能都是基于GPIO口的比较容易。

摇头避障功能更多是C51单片机关于定时器的使用，摇头功能是舵机SG90实现的，用GPIO配合定时器来输出PWM波形控制舵机角度，障碍物检测用的超声波

模块，需要根据模块的时序图开发，计算波发出后，遇到障碍物再返回的总时长，来获取障碍物距离。

而测速功能也是定时器配合外部中断来实现，借用小车码盘，使用外部中断来计数，车轮每转过一圈就触发一次中断，这样可以控制定时器1秒时间内的圈数，

从而计算出小车速度。

最后一个功能是模式切换，因为小车没有同时运行在寻迹，避障，跟随等模式，那么我通过串口外接一个语音模块，根据语音模块固件对声音处理的结果，

通过读取串口数据来切换工作模式。

6.3 刁难型问题

面试官刁难小车无非有两个目的：

- 面试官本人技术也很一般，需要在新人面前找存在感，一般体现在年轻面试官身上

- 面试官技术可以，老技术，不会看不起新人的技术位置，更多想看你怎么现场化解矛盾和尴尬，对你的表达能力做考验

6.3.1 你这小车就是玩具，没有应用场景

你说的是对的，小车就是玩具，新人在没有入行前，比如往届电赛，或者电协的同学经常会玩，现在我的技术能力也不仅限于小车，回头来看这个小车的话

确实也称不上是个项目，更多是C51单片机的学习总结，前面做功能实现介绍的时候，其实每个功能背后都是单片机技术组成，比如GPIO，中断，串口，

涉及很多芯片手册中对寄存器的理解，对外接模块时序图的分析，我觉得这些能力是独立于小车存在的，在工作中如果给我一个开发任务，需要配置寄存器，

需要对时序去驱动一些功能的话，本质运用的软硬件开发调试能力是可以平移的。

比如我使用共享单车的时候就会去猜测自行车上面的硬件架构，因为扫码开锁的时候提醒手机要开启蓝牙，那么共享单车的硬件方案中是不是存在蓝牙模块，

对蓝牙模块的通信是不是用串口，还是GPS定位模式很多也是串口协议的，对于MCU串口驱动配置，及串口数据解析在我做小车的时候都是有干过的。

所以你说的对，小车确实是玩具，但背后的嵌入式开发技术可以沿用到后续工作中，至少我在二次学习或者开发过程中的反应应该会很快的。

6.3.2 小车都被做烂了，怎么还写小车

小车被做烂了是事实，之所以那么多人做小车，我觉得还是关于小车的资料是很多的，很多电赛也在玩小车。

我们从0开学学习嵌入式的时候，难免会去做这个东西

小车也好，智能门锁也吧，背后其实都是C语言，芯片手册，时序图，软硬件联调的技术和能力锻炼，而我今天能来面试，绝不可能妄想通过小车来获得

一份工作，这个案例是我在刚开始学习嵌入式的时候做的，其实LOW归LOW，但还是让我在当时那个阶段有所收获的。